



Instituto Nacional Técnico y Tecnológico
Centro Tecnológico “Taller Luban”
Nicaragua - INATEC

Curso: Fundamentos de Programación con Python

Clase #3:

Estructuras de Datos

- Contenido:**
- ¿Qué son las estructuras de datos?
 - Duplas
 - Ejemplos
 - Ejercicios prácticos

Docente: Ing. Fabiola Cerrato

Estructuras de Datos: ¿Qué son?

Las **estructuras de datos** o **secuencias** se caracterizan por ser una concatenación de elementos (del mismo tipo o de tipos distintos) dispuestos de forma secuencial que permiten hacer operaciones de forma simple e intuitiva.

De forma abstracta y simple, se podría decir que los tipos secuencia son como un mueble que tiene **N** cajones, y todos están dispuestos uno detrás de otro. Cada cajón puede almacenar un tipo distinto de elemento y, según la naturaleza del mueble, que posee unas características propias, se podría expandir dinámicamente o tendría una longitud fija o se podría iterar de una determinada manera. (JÍMENEZ, 2021)

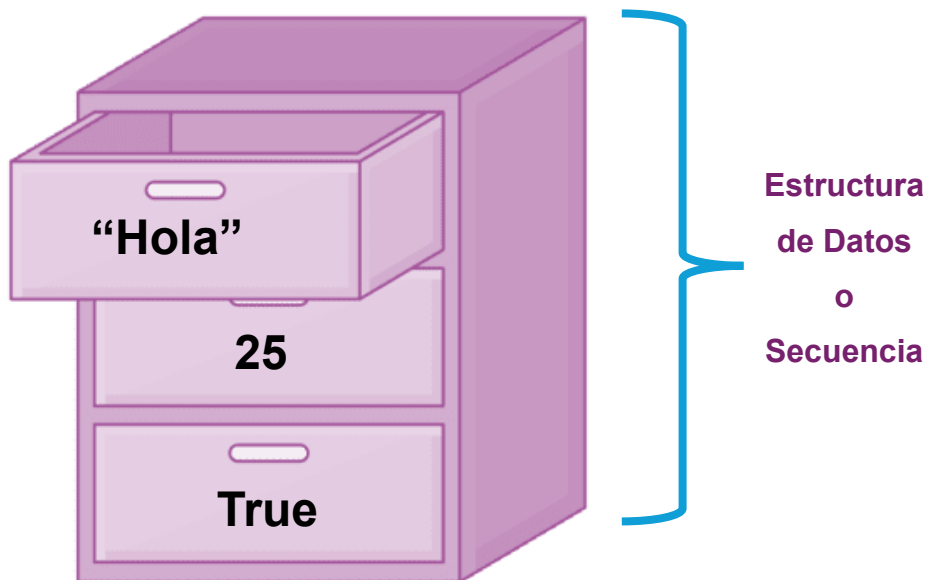
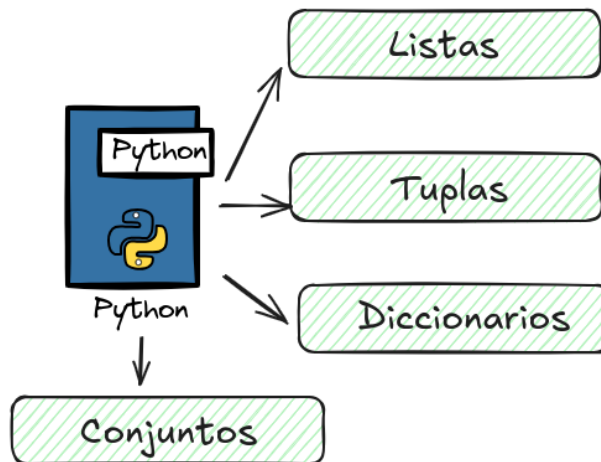


Ilustración 1: Ejemplo figurado de estructura o secuencia de datos

Tipos de secuencias



Estructura de datos #1: Duplas



¿Qué son?

Una tupla en Python es una colección de elementos ordenados e **inmutables** que se utiliza para almacenar varios elementos en una sola variable. Una vez creada, no puedes cambiar ni modificar los elementos. (GmbH, s.f.)

Tiene dos características principales:



Ilustración 2: Características de las tuplas

Ejemplo:

Nombre de la Tupla Elementos de la Tupla

objetos = (7, 'Hola', True, 3.5)

Ilustración 3: Ejemplo de tupla en Python

Nota: El constructor de la tupla es la coma, no el paréntesis, pero el intérprete muestra los paréntesis, y nosotros deberíamos utilizarlos, por claridad.

Además, hay que tener en cuenta que es necesario añadir una coma para tuplas de un solo elemento, para diferenciarlo de un elemento entre paréntesis.

Ejemplo:

```
frutas = ("manzana")
```

```
type(frutas) # SALIDA: type "Str"
```

```
frutas = ("manzana",) # SALIDA: type "tuple"
```



Ejemplo:

Se tiene la siguiente tupla la cual representa las notas de unos estudiantes:

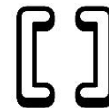
```
Notas_estudiantes = (90, 58, 77, 81, 92, 65, 87)
```

1. Si deseamos **imprimir toda la tupla** podemos hacer lo siguiente:

```
print("Las Notas de los estudiantes son:", Notas_estudiantes)
# SALIDA: Las Notas de los estudiantes son: (90, 58, 77, 81, 92, 65, 87)
```

2. Si deseamos **imprimir una nota en particular** debemos saber qué posición ocupa el elemento en la tupla.

Para referirnos a elementos de una tupla, se usa el operador:



Cada elemento de la tupla tiene una posición específica que empiezan desde la posición 0.

```

                [0]  [1]  [2]  [3]  [4]  [5]  [6]
Notas_estudiantes = (90, 58, 77, 81, 92, 65, 87)

```

Entonces si deseamos imprimir la nota del cuarto estudiante debemos pasar el índice 3:

```

print("La cuarta nota es:", Notas_estudiantes[3])
# Salida: La cuarta nota es: 81

```

También podemos utilizar índices negativos para indicarle al compilador que deseamos acceder a los elementos empezando desde el final:

Forma normal: índice positivo

```

print("La última nota es:", Notas_estudiantes[6])
# Salida: La última nota es: 87

```

Forma con índice negativo:

```

print("La última nota es:", Notas_estudiantes[-1])
# Salida: La última nota es: 87

```

3. Acceder a **rangos de elementos** de una tupla

Si deseamos mostrar cierta parte de una tupla podemos utilizar dentro de los corchetes dos puntos y un número entero que representará la cantidad de elementos a mostrar de la tupla.

Estructura:

Nombre_tupla[inicio:fin:paso]

inicio: Índice donde comienza

fin: Índice donde termina

paso: Saltos entre elementos
(opcional, default=1)

Ejemplos:

```
Notas_estudiantes = (90, 58, 77, 81, 92, 65, 87)
```

```
print("Las tres primeras notas son:", Notas_estudiantes[:3])
```

```
# Salida: Las tres primeras notas son: (90, 58, 77)
```

```
print("Las tres últimas notas son:", Notas_estudiantes[-3:])
```

```
# Salida: Las tres últimas notas son: (92, 65, 87)
```

```
print("Las notas desde la segunda hasta la cuarta son:",
```

```
Notas_estudiantes[1:4])
```

```
# Salida: Las notas desde la segunda hasta la cuarta son: (58, 77, 81)
```

```
print(Notas_estudiantes[1:6:2])
```

```
# Salida: (58, 81, 65)
```

Nota: El número del índice de **fin** trabaja igual que la función `range()`, terminando 1 índice antes.

Métodos para Tuplas

1. Conteo de elementos en la tupla:

Para saber la cantidad de elementos en una tupla utilizamos el método ***len()*** el cual es perteneciente a la clase de los tipos cadenas:

```
print("Número de elementos en la tupla:", len(Notas_estudiantes))
```

```
# Salida: Las notas desde la segunda hasta la cuarta son: (58, 77, 81)
```

2. Conocer **índices** de elementos en la tupla:

Para conocer el índice de los elementos en una tupla utilizamos el método ***index()*** al cual debemos de pasarle como argumento el elemento a retornar índice:

```
print("Índice de la nota 92:", Notas_estudiantes.index(92))
# Salida: Índice de la nota 92: 4
```

3. **Tamaño** de una tupla

Para saber la cantidad de veces en que se repiten elementos en una tupla llamamos al método ***count()***

```
print("Número de veces que aparece la nota 81 es:", Notas_estudiantes.count(81))
# Salida: Número de veces que aparece la nota 81: 1
```

Búsqueda de elementos en la tupla

En el inciso anterior vimos el método ***count()*** para buscar un elemento específico en la tupla. Sin embargo, también tenemos el operador ***in*** para realizar la misma operación:

Ejemplo 1:

```
Notas_estudiantes = (90, 58, 77, 81, 92, 65, 87)
nota_a_buscar = 77

if nota_a_buscar in Notas_estudiantes:
    print(f"La nota {nota_a_buscar} se encuentra en la tupla.")
else:
    print(f"La nota {nota_a_buscar} no se encuentra en la tupla.")

# Salida: La nota 77 se encuentra en la tupla.
```


Ejemplo 2:

```
Notas_estudiantes = (90, 58, 77, 81, 92, 65, 87)

nota_a_buscar = 40

if nota_a_buscar in Notas_estudiantes:
    print(f"La nota {nota_a_buscar} se encuentra en la tupla.")
else:
    print(f"La nota {nota_a_buscar} no se encuentra en la tupla.")
```

Salida: La nota 40 no se encuentra en la tupla.



A pesar de no poder modificar una tupla una vez creada, si podemos fusionar más de una. Para ello podemos utilizar el operador: +

Ejemplo:

```
Notas_estudiantes = (90, 58, 77, 81, 92, 65, 87)
nuevas_notas = (75, 88)
notas_actualizadas = Notas_estudiantes + nuevas_notas
print("La Notas han sido actualizadas:", notas_actualizadas)
```

Salida:

La Notas han sido actualizadas: (90, 58, 77, 81, 92, 65, 87, 75, 88)



Duplicado de tuplas

Las tuplas no tienen un método específico como otras estructuras para realizar esta tarea; sin embargo, puede realizarse de forma manual creando una nueva variable y asignándole la tupla con los índices implícitos.

Ejemplo:

```
Notas_estudiantes = (90, 58, 77, 81, 92, 65, 87)
copia = Notas_estudiantes[:]
print(copia)
```

```
# Salida: (90, 58, 77, 81, 92, 65, 87)
```

Ejercicios Prácticos

1. Resuelva cada uno de los siguientes escenarios poniendo en práctica lo aprendido en clases:

a) Se tiene la siguiente tupla: `mi_tupla = [10, 20, 30, 40, 50, 60, 70, 80]`

Acceder al primer elemento

Obtener a los últimos cuatro elementos

Acceder a los elementos de los índices 3 al 6

b) Se tiene la siguiente tupla: `factura = ['pan', 'huevos', 100, 1234]`

Tamaño de la tupla

Obtener el índice del elemento 'huevos'

```
# Buscar si existe el elemento 'Coca cola'
```

2. Encuentre el error en cada uno de los siguientes escenarios:

a) El siguiente código intenta modificar una tupla:

```
colores = ("rojo", "verde", "azul")
colores[1] = "amarillo"
print(colores)
```

b) El siguiente código intenta imprimir los elementos de una tupla:

```
datos = ("Juan", 30, "Ingeniero")
nombre, edad = datos # Asignando etiquetas a los elementos
print(f"Nombre: {nombre}, Edad: {edad}, Profesión: {profesion}")
```

c) El siguiente código intenta imprimir elementos de la tupla:

```
frutas = ("manzana", "pera", "naranja", "plátano", "uva")
print(frutas[5])
print(frutas[-6])
frutas[1:3] = ("kiwi", "mango")
```

Ejercicios Propuestos

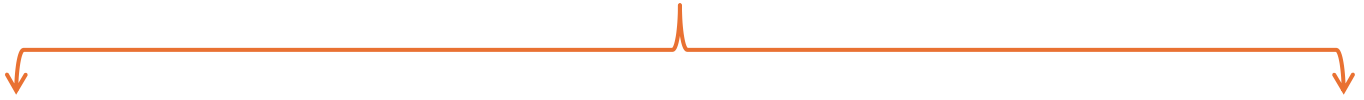
NIVEL FÁCIL



1. Crea una tupla con los días de la semana y muestre el tercer día.
2. Cree una tupla de 10 números enteros, muestre cuántos elementos tiene.
3. Cree 2 tuplas con 5 elementos cada una (con tipos de datos variados), fusiónelas y muestre la fusionada por pantalla.
4. Cree una tupla con cadenas de texto, pida una cadena por teclado y verifique si existe en la tupla.
5. Dada la siguiente tupla, pida un número entero por teclado y cuente cuántas veces aparece el número en la tupla:

```
tupla_numerica = (25, 38, 12, 5, 22, 69, 81, 45, 12, 28)
```

NIVEL INTERMEDIO



6. Pida 5 datos por teclado, almacénelas en una tupla y luego muestre todos los datos de la tupla por pantalla.
7. Pida 8 nombres de animales, almacénelas en una tupla y luego muestre por pantalla cuántas veces se repitió cada uno.
8. Pida al usuario un número N, luego pida números enteros según la cantidad N y almacénelos en una tupla, luego recorra la tupla para guardar en otra tupla los números pares y en otra los impares. Muestre por pantalla las 3 tuplas.

NIVEL DIFÍCIL



9. Pida al usuario un número N el cual representará la cantidad de tuplas a crear. Por cada tupla pida 5 datos y almacénelos en su tupla correspondiente. Luego muestre por pantalla un menú de opciones en el cual permita:
 - a) Mostrar una tupla específica
 - b) Buscar un valor en una tupla
 - c) Mostrar tamaño de una tupla
 - d) Fusionar todas las tuplas
 - e) Salir